Information and Software Technology 54 (2012) 417–431

Contents lists available at SciVerse ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

A framework for analysis and design of software reference architectures

Samuil Angelov [a,*], Paul Grefen [a], Danny Greefhorst [b]

[a] School of Industrial Engineering, Eindhoven University of Technology, P.O. Box 513, 5600 MB Eindhoven, The Netherlands
[b] ArchiXL, Nijverheidsweg Noord 60-27, 3812 PM Amersfoort, The Netherlands

ARTICLE INFO

Article history:
Received 23 October 2010
Received in revised form 29 November 2011
Accepted 29 November 2011
Available online 7 December 2011

Keywords:
Software reference architecture
Software domain architecture
Software architecture design
Software product line architecture

ABSTRACT

Context: A software reference architecture is a generic architecture for a class of systems that is used as a foundation for the design of concrete architectures from this class. The generic nature of reference architectures leads to a less defined architecture design and application contexts, which makes the architecture goal definition and architecture design non-trivial steps, rooted in uncertainty.
Objective: The paper presents a structured and comprehensive study on the congruence between context, goals, and design of software reference architectures. It proposes a tool for the design of congruent reference architectures and for the analysis of the level of congruence of existing reference architectures.
Method: We define a framework for congruent reference architectures. The framework is based on state of the art results from literature and practice. We validate our framework and its quality as analytical tool by applying it for the analysis of 24 reference architectures. The conclusions from our analysis are compared to the opinions of experts on these reference architectures documented in literature and dedicated communication.
Results: Our framework consists of a multi-dimensional classification space and of five types of reference architectures that are formed by combining specific values from the multi-dimensional classification space. Reference architectures that can be classified in one of these types have better chances to become a success. The validation of our framework confirms its quality as a tool for the analysis of the congruence of software reference architectures.
Conclusion: This paper facilitates software architects and scientists in the inception, design, and application of congruent software reference architectures. The application of the tool improves the chance for success of a reference architecture.

1. Introduction

Software reference architectures have emerged as abstractions of concrete software architectures from a certain domain. A reference architecture (RA)[1] is used for the design of concrete architectures in multiple contexts serving as an inspiration or standardization tool [1,2]. Nowadays, the increasing complexity of software, the need for efficient and effective software design processes and for high levels of system interoperability lead to an increasing role of reference architectures in the software design process.

Concrete software architectures are designed in a specific context and reflect concrete business goals of the stakeholders. The types of possible goals, the identification of the stakeholders, the identification of the required system functionalities and qualities, and their effects on the architecture design have been well-studied and extensively published (see, e.g., [3,4]). Reference architectures, however, are designed to facilitate system design and development in multiple projects. Consequently, their design and application take place in a broader and, hence, less-defined context with a larger and less-defined stakeholder base. These less defined architecture design and application contexts and stakeholders, affect the ability of the architecture sponsors[2] and designers for clear judgment and decision making when articulating the architecture goals and elaborating the architecture design [3,4].

The reference architecture design and application contexts, goals, and design exist in a complex relation (see Fig. 1). The architecture goals set constraints on the context in which the architecture should be defined and on the architecture design. Vice versa, the context and design affect the achievement of the architecture goals. Furthermore, the design choices are affected by the context and vice versa, a design choice implies a certain context. In this

* Corresponding author. Address: Information Systems Group, School of Industrial Engineering, Eindhoven University of Technology, P.O. Box 513, 5600 MB Eindhoven, The Netherlands. Tel.: +31 40 247 2617; fax: +31 40 243 2612.
E-mail addresses: s.angelov@tue.nl (S. Angelov), p.w.p.j.grefen@tue.nl (P. Grefen), dgreefhorst@archixl.nl (D. Greefhorst).

[1] Throughout the paper, we use the term "reference architecture" to refer to the documented description of a software reference architecture.

[2] The stakeholders initiating the design of the architecture.

0950-5849/$ - see front matter © 2011 Elsevier B.V. All rights reserved.
doi:10.1016/j.infsof.2011.11.009

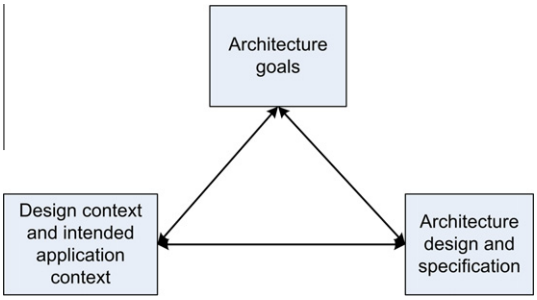

Fig. 1. The relationships between architecture goals, design and contexts.

work, we investigate these relationships and the ways they influence the success of a reference architecture (where by success, we mean the acceptance of the architecture by its stakeholders and its usage in multiple projects).

We call a reference architecture "congruent" if its goals are relevant for the context of the reference architecture and its design properly reflects both the architecture context and goals, i.e., we require congruence between context, goals, and design of a reference architecture. To address the problem of designing congruent reference architectures, in this paper, we propose a "framework for congruent reference architectures". The framework consists of two elements, i.e., a multi-dimensional architecture classification space and a set of predefined reference architecture types (and variants of these types). The multi-dimensional space allows classification of reference architectures according to their context, goals, and design. The reference architecture types are specific combinations of values from the multi-dimensional space. Reference architectures that fit into one of these types are congruent and, as we show in this work, have higher chances for success. Lack of a fit into a type indicates an incongruent architecture and is a portent for the architecture weak usage and criticism from the architecture stakeholders.

The framework that we propose serves multiple purposes. It can be applied for the analysis of the congruence of existing reference architectures. Furthermore, it can be used as a "tool" for the design of congruent reference architectures as well as for the re-design of existing reference architectures into congruent reference architectures.

The work presented in this paper is a follow-up of the work presented in [5] – it substantially extends and improves the results presented there. In the current paper, we include a comprehensive discussion on the concept of reference architecture to better define it and provide a clear disambiguation with respect to similar concepts. We present a more detailed and precise (and, hence, more operational) description of the framework for classifying reference architectures. Furthermore, we provide a description of the framework usage. The validation of the framework is tentative in earlier work – in the current work it is performed in a more profound way and the set of reference architectures used in the validation is extended.

The structure of this paper is as follows. The lack of an accepted definition for the term "reference architecture" by the research community has led to a semantic overloading of the term. Therefore, to position clearly our work, we start with defining the term "reference architecture" in Section 2 and discuss it with respect to a number of closely related terms. Section 3 and 4 contain the description of our framework. In Section 3, we describe the multi-dimensional space for classification of reference architectures. In Section 4, we present the predefined types of reference architectures. In Section 5, we discuss how the framework can be used. In Section 6, we validate the framework by applying it for the analysis of 24 reference architectures. Section 7 contains a discussion on related work. The paper ends with conclusions.

2. Reference architectures – a definition and disambiguation

A commonly accepted definition for software reference architectures does not exist [6–8]. In this section, we first provide the definition of reference architectures that we use throughout the paper. Next, from the perspective of this definition, we discuss a number of terms that are sometimes considered synonymous or closely related to the term "reference architecture". We present our view on the overlap and differences in the semantics of these terms.

2.1. A definition for reference architectures

In [3], a reference model is defined as "*a division of functionality together with data flow between the pieces*" and a reference architecture as "*a reference model mapped onto software elements (that cooperatively implement the functionality defined in the reference model) and the data flows between them*".

We have selected this definition for our work as it addresses the generic nature of a reference architecture, states explicitly its software nature, defines the minimum architectural elements that should be used in its design, and covers the various types of software reference architectures that exist in practice. Other definitions cover not only software but also business and technology aspects (e.g., [9]), do not require the usage of specific architectural elements (e.g., [10]), or restrict too strongly the space of reference architectures (e.g., focusing primarily on best-practices architectures [7,8]). More importantly, while other definitions implicitly or explicitly equate the term "reference architecture" to other related terms, this definition allows us to clearly delineate the borders of the term "reference architecture". Next, we discuss the relation of a number of terms to the term "reference architecture" from the perspective of our definition.

2.2. Reference architectures and concrete architectures

Typically, an architecture is designed and used for the development of a specific software application. We call this type of architectures "concrete architectures" or simply "architectures" (also known as "application architectures" [11]). What if a concrete architecture is used for the design of several applications? Does it become a reference architecture? In other words: Where is the border between concrete and reference architectures?

In [7,8,1] we have identified the generic nature of reference architectures as a main feature distinguishing them from concrete architectures. Their generic nature implies their applicability in multiple, different contexts, reflecting the requirements of the stakeholders in these contexts. The generic nature of reference architectures is achieved by designing them at higher levels of abstraction (abstracting from the differences introduced by the contexts). Thus, we can label an architecture as reference, only if it is defined to abstract from certain contextual specifics allowing its usage in differing contexts.

2.3. Reference architectures and architectural patterns

Architectural patterns are a widely used concept in the software architecture community. According to Avgeriou and Zdun [12], architectural patterns are "well-established solutions to architectural problems". In [13], an architectural pattern is defined as a construct that "expresses a fundamental structural organization or schema for software systems. It provides a set of predefined subsystems, specifies their responsibilities, and includes rules and guidelines for organizing the relationships between them". In [3], an architectural pattern is defined as "a description of element

and relation types together with a set of constraints on how they may be used".

In a number of publications, reference architectures are equated to architectural patterns. In [9], a reference architecture is defined as "a predefined architectural pattern, or set of patterns, possibly partially or completely instantiated, designed, and proven for use in particular business and technical contexts, together with supporting artifacts to enable their use". In [14], reference architectures are seen as "proven patterns of how things do work". In [6], the authors consider architectural patterns as reference architectures that have a very narrow coverage and point out that reference architectures may have more extended coverage. However, architectural patterns are "domain-independent" and designed to address certain architectural qualities (e.g., performance, interoperability). In contrast, the focus of reference architectures lies in defining the functionalities required in a domain (defined in the corresponding reference model) and their interaction with the domain environment. These two differences between architectural patterns and reference architectures clearly separate the two concepts. As stated in [3], architectural patterns can be used in the design of reference architectures for achieving desired architectural qualities (see Fig. 2).

2.4. Reference architectures and software product line architectures

Software product line architectures (also called shortly product line architectures [15]) and reference architectures are often equated. In [16], these terms are considered "similar" and "used interchangeably" and in [17,18] they are considered equivalent. In [19], a reference architecture is defined as the "core architecture for subsequent product families (and products) independently of implementation". A similar view is expressed in [20,21], and [22]. Conversely, in [4], reference architectures are separated from product line architectures, and are viewed as more generic and potentially less complete architectures.

The definition of reference architectures that we use in this paper covers the definition of software product line architectures. Thus, in our understanding, software product line architectures are reference architectures. However, not all reference architectures are software product line architectures (as also noted in [4]). Product line architectures specifically address points of variability – they define extension points at which specific behaviors can be instrumented. Product line architectures are defined with the clear vision for their realization in a platform that will support a range of products within an organization. Furthermore, they tend to have more formal specification in order to ensure clear and precise behavior specifications at well-specified extension points [23]. These specifics of product line architectures are not necessarily present in all reference architectures. Many reference architectures are defined at a more abstract level and do not to serve for the design and development of software product families (e.g., [24–26]). Thus, in our view, software product line architectures are a subset of the set of reference architectures.

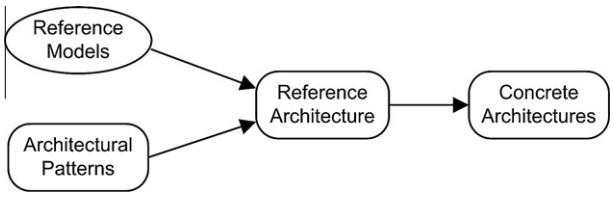

Fig. 2. The relationship between reference models, architectural patterns, reference architectures, and concrete architectures (adapted from [3], arrows indicate model data input).

Product line architectures are defined with a specific goal in mind, i.e., the realization of the family of software products, and in a context that is typically more specific than generally for reference architectures (the organization that produces the software products is typically known a priori). Consequently, obtaining congruence for product line architectures is in general easier than for general reference architectures. Furthermore, product line architectures have been in the focus of numerous publications (as we discuss in Sections 4.1 and 7). This accumulated knowledge and experience additionally facilitates design of congruent PLA.

2.5. Reference architectures and domain-specific software architectures

In the early 1990s, the concept of domain-specific software architectures (DSSA) was coined and researched in a number of DoD funded projects [27]. A DSSA (illustrated in [28]) is defined as a collection of a domain model, reference requirements, a reference architecture, a supporting infrastructure, and an instantiation/refinement process [11]. The definition provided in [11] implies that the concept of DSSA encompasses a set of artifacts among which reference architectures. However, the term DSSA has been associated in literature mainly with the "reference architecture" element of DSSA [4,17,22,29].

The explanations on DSSA provided in [11] refer several times to the concept of product line architectures, although DSSA do not explicitly target product line architectures. Furthermore, a "domain" in DSSA is defined as "a set of 'common' problems or functions that applications in that domain can solve/do" [11] and "a particular type of task" [30]. This open definition, based on the notion of common functionality, together with the notion of domain generalization [28], allows for a broad interpretation of the "domain" concept in DSSA. The broad definitions and explanations have led to different interpretations of the "reference architecture" element of DSSA. In [17,22,29], a DSSA is viewed as a product line software architecture where the domain is perceived from the perspective of the organization that will produce the related software products. Contrary, in [4], DSSA are positioned as generic architectures, with the domain being a general, (non-organization or product related) problem space (defined on the basis of common functionalities of applications in this domain). This perspective on DSSA overlaps with our definition of reference architectures.

To summarize, in our work, we cover both interpretations of the "reference architecture" element of DSSA, i.e., as a software product line architecture (see Section 2.4) and as a generic architecture scoped by common functionalities of applications.

2.6. Structured and non-structured reference architectures

In [10], the term "non-structured reference architectures" is used to denote reference architectures that comprise a set of architectural rules (principles) which should be satisfied in the design of a concrete architecture. The main argument for designing reference architectures without the specification of software components and connectors is that "an industry-level reference architecture cannot prescribe too much structure as it may prevent new business relationships/models among different parties and adoption of new technologies". In [7], the authors accept the possibility for the lack of explicit structure in a reference architectures as well. The Dutch Government Reference Architecture (NORA) [31] and the LIXI reference architectures [32] are examples of "non-structured reference architectures".

"Non-structured reference architectures" fall outside the scope of the definition for software reference architectures that we use in this paper, as they only specify the underlying principles of a reference architecture but do not address its actual design (its

software elements and data flows between them). The fundamental role of the structure in reference architectures given by components and connectors is also acknowledged by practitioners [8]. We see "non-structured reference architectures" as one of the deliverables elaborated before the design of reference architectures (i.e., "ante-reference-architectures").

3. The multi-dimensional space for reference architectures

In this section, we present the multi-dimensional space for the classification of reference architectures. The space is constructed with the aim to support analysis of reference architectures in terms of relationships between their context, goals, and architecture design (see Fig. 1). Consequently, it is based on three dimensions: *context*, *goals*, and *design*. For each dimension, we define orthogonal sub-dimensions that address a specific aspect of the dimension. We have used the dimensions for concrete architectures defined in [33,34] for the definition of our sub-dimensions. In addition, we have used industry experiences and research results on classifications of reference architectures [1,7,8,2].

We denote the set of sub-dimensions by means of interrogatives in order to make them more intuitive for the reader (the usage of interrogatives is a well-known technique for high-level problem analysis [35], for example applied in the well-known Zachman framework [36]). We use the Where, Who and When interrogatives to address the context sub-dimensions, as these questions refer to the description of a contextual information. The How and What refer to operational aspects and are used in the design dimension. We use the Why interrogative in the goal dimension.

3.1. Context dimension and its sub-dimensions (C)

In this dimension, we define the aspects of the design and application contexts which may affect the goals and design of a reference architecture.

3.1.1. C1: Where will it be used?

In [33], a "stakeholder" dimension is defined that describes the stakeholders of the architecture application. We use this dimension to address on a coarse-grained level the organizations that can be the intended recipients of the reference architecture, i.e., the organizations at which the reference architecture is intended to be applied for the design of concrete architectures. A reference architecture can be designed with an intended scope of a *single organization* or *multiple organizations* that share a certain property (they may share a market domain or a geographical property such as a country). Hence, the C1 sub-dimension contains two values: *single organization* and *multiple organizations*.

3.1.2. C2: Who defines it?

We use the "stakeholder" dimension from [33] as an inspiration to define its equivalent dimension in the context of the architecture design. The C2 sub-dimension lists the stakeholders that can be involved in the design of a reference architecture. In the case of multiple organizations involved in the design process, the stakeholders are the types of organizations that can be involved in the process. These can be *software organizations* (designing or applying a reference architecture), *user organizations* (users of the software solutions based on a reference architecture), and *independent organizations* (organizations that do not implement reference architectures for the design of software solutions or use these solutions). Independent organizations can be, for example, *research centers*, *standardization organizations*, *non-profit organizations*, *governmental,* and *non-governmental organizations*. In the case of single organization involved in the design process, the stakeholders are the analogous organizational groups (i.e., departments, business units): *software groups*, *user groups*, and *independent groups* (e.g., *management groups*). On a lower level of granularity, the stakeholders can be described through the types of people that may participate in the design process, i.e., software designers, software users, software researchers, software/project managers (which can be further specialized as discussed in [37]).

Similar to the design of concrete architectures, in the design of a reference architecture there should be stakeholders that provide requirements and that elaborate the architecture design. In the sequel of this paper, we denote these roles of the stakeholders as "R" (Requirements providers) and "D" (Designers), respectively. Note that a stakeholder may act in both roles.

3.1.3. C3: When is it defined?

The C3 sub-dimension addresses timing issues that may have an effect on the goals and design of a reference architecture. A reference architecture can be designed before any existing commercial system or a set of commercial systems together fully implement the reference architecture or after experience from commercial application has already been accumulated [1]. Hence, we define two possible values for this sub-dimension, i.e., *preliminary* and *classical* reference architectures. This dimension is inspired by the "transformation" dimension in [33].

A *preliminary reference architecture* is a reference architecture that is defined when the technology, software solutions, or algorithms demanded for its application do not yet exist in practice by the time of its design. They may exist only as research experiments, may be developed specifically to support the usage of the reference architecture or may be partially, or even fully, non-existent. Typically, this is the case when a reference architecture is defined before any concrete systems in the specific domain exist in practice. But note that even if concrete systems do exist, designers may still opt for the design of a preliminary architecture, thereby providing an innovative design with respect to the existing state of the art.

A *classical reference architecture* is a reference architecture that is defined when technology, software, and algorithms required for the architecture application exist by the time of its design and have been tested in practice. Classical architectures take the practical experience with the use of concrete commercial systems into account in its definition. As such, one might paraphrase a classical reference architecture as a "best practice reference architecture". For the design of a proper classical reference architecture, a substantial base of experience must be available with preferably a number of different concrete systems in the class that the reference architecture describes.

3.2. Goal dimension and its sub-dimensions (G)

In the goal dimension, we define the possible goals of a reference architecture. For the goal dimension we define only one sub-dimension (which can be seen as coinciding with the main goal dimension). This sub-dimension represents the general goal of a reference architecture that has to be aligned with the architecture contexts and design. Goals can be decomposed into sub-goals, reaching detailed goal specification in terms of desired architecture, business, and system quality attributes [3,37]. However, discussion of architecture congruency in terms of detailed goals results in detailed and fragmented requirements on the context and design dimensions and cannot deliver the overall, general framework for congruency that we aim at. That is why we focus on one general goal dimension in our multi-dimensional space.

3.2.1. G1: Why is it defined?

Based on [2], we distinguish two possible values for the goal of a reference architecture: *standardization* of concrete architectures (aiming at system/component interoperability) and *facilitation* of the design of concrete architectures (aiming at providing guidelines and inspiration for the design of systems). Different types of standardization and facilitation goals can be distinguished [2]. However, these lower level goals do not interplay differently with the context dimensions (hence, they do not have to be addressed in our classification space). This dimension is related to the "nature" dimension in [33].

3.3. Design sub-dimensions (D)

The sub-dimensions in this dimension describe a reference architecture in terms of its "operational" side, i.e., its design and specification. We have identified four relevant sub-dimensions. These sub-dimensions address the contents of a reference architecture, its level of detail, its level of concreteness, and the techniques used for its representation. The first sub-dimension corresponds to the *What* interrogative, the latter three together correspond to the *How* interrogative.

3.3.1. D1: What is described?

This sub-dimension lists the elements that can be defined in a reference architecture. It is based on the "type of information" dimension in [33], tailored for the context of reference architectures. As element types in a reference architecture, we distinguish *components and connectors* (based on the definition we use, these elements should be present in any reference architecture), *interfaces, protocols, algorithms* and *policies and guidelines.* The *policies and guidelines* are also sometimes referred to as "texture" [18,21].

3.3.2. D2: How detailed is it described?

This sub-dimension lists possible levels of detail at which the elements of a reference architectures (see D1 sub-dimension) can be defined, i.e., it corresponds with an aggregation dimension [21]. The sub-dimension is based on the "detail level" dimension in [33]. Similar to [33], we distinguish three levels of detail, i.e., *detailed*, *semi-detailed*, and *aggregated* specification of the elements of a reference architectures. These detail levels can be defined for each of the values defined in D1 (e.g., detailed components, semi-detailed guidelines).

Even though this sub-dimension is perhaps intuitively easy to understand, it is not easy to provide generally applicable, operational guidelines for the assignment of values in this dimension. These guidelines are context- and domain-specific: in some contexts or domains, even aggregated architectures may be quite detailed (in the sense that they contain many elements) because the context or domain is essentially complex in functionality (or contains many repetitive elements).

One obvious method to "measure" the level of detail is counting the number of elements in (a part of) a reference architecture (taking the above caveat into account). A second method is counting the number of explicit aggregation levels used for the specification of the reference architecture (also advocated in [38]). To give an example of the latter, we can compare the component description of two reference architectures for workflow management systems. The WRM [26] contains one aggregation level for the software component specification, whereas the Mercurius architecture [22] contains three aggregation levels. Hence, we may state that the component specification of the WRM is aggregated and that of Mercurius is detailed.

In this paper, we use both methods to classify reference architectures in the aggregation dimension. Reference architectures containing numerous elements described at more than two aggregation levels belong to one end of the dimension (i.e., *detailed architectures*), while architectures having few elements and defined at a single level of aggregation belong to the other end of the dimension (i.e., *aggregated architectures*). *Semi-detailed architectures* lie in the "gray" area between these two ends. This classification technique, although imprecise, suffices for the goals of our framework.

3.3.3. D3: How concrete is it described?

This sub-dimension lists the possible levels of abstraction of a reference architecture. Abstraction is related to the level of choices made in an architecture in terms of technology, applications, vendors, etc. This sub-dimension is related to the "abstraction" dimension in [34]. We limit our values in this sub-dimension to *abstract, semi-concrete,* and *concrete* (in [34], four abstraction levels are explained, but this refers to a broader spectrum of architectures including concrete architectures). Similarly to D2, the values of D3 can be defined for each value of D1.

An *abstract* (part of a) reference architecture is specified in terms that very generally specify the character (e.g., functionality) of an element of that architecture. It leaves the specific choice for a specific element fully open. A *semi-concrete* reference architecture is defined in terms that clearly specify a specific class of options for each element in the architecture. A *concrete* reference architecture specifies the choice from the class of options for each element. As an example, an abstract reference architecture may indicate that a software component is a "data management module", whereas a semi-concrete reference architecture may use the specification "database management system" and a concrete reference architecture "corporate relational database management system".

3.3.4. D4: How is it represented?

The D4 sub-dimension distinguishes between the possible levels of formalization of reference architectures. It is based on the "representation" dimension in [33] and uses the same three levels defined there: *informal*, *semi-formal, formal.* Again, the values of D4 can be defined for each value of D1.

Following [20], the three values can be operationalized as follows. An *informal* representation of (a part of) a reference architecture typically uses natural language (or an ill-defined graphical notation). Obviously, an informal representation leaves much room for interpretation and hence ambiguity but requires also no architecture formalization knowledge from the stakeholders and users. A *semi-formal* representation is based on a well-defined notation, such as UML, that typically has semantics that is clearly defined but does not have a complete formal (mathematical background). Thus, it is a reasonable compromise between clarity and precision and required architecture formalization knowledge from the stakeholders. Semantics of a semi-formal representation is much stricter than of an informal representation, but ambiguity is still possible. A *formal* representation is based on a formal architecture specification language such as C2 or Rapide [39], which has a complete mathematical underpinning and hence strictly defined semantics.

4. Types of reference architectures

The problem of identifying the types of reference architectures is a simple constraint-satisfaction problem in which combinations of values of the sub-dimensions should satisfy a number of constraints. The constraints that we have used for the selection of values of the sub-dimensions are postulates based on existing literature on reference architectures, our experience (including the one gained from the 16 reference architectures that we have examined in our explorative study [5]), and "common-sense" reasoning. These postulates are presented in this section as part of the

description of the architecture types. We have used a backtracking algorithm for the problem analysis. We started by fixing the value for the goal dimension and discussed the possible context and design values based on the goal dimension value. Next, we present our results on the types of reference architectures, first for the *standardization* goal value, then for the *facilitation* goal value.

Note that the values of a reference architecture are mutually exclusive for the G1, C1, and C3 sub-dimension (i.e., a reference architecture can be attributed only one value from these sub-dimensions) and mutually inclusive for the C2 sub-dimension (multiple values from the sub-dimension can be attributed to an architecture). As we show next, in some cases, different combinations of values for the C2 sub-dimension are possible for a fixed set of values from the other sub-dimensions. These variations in the C2 sub-dimension have limited effects on the design dimensions. That is why, when such variations are possible, we use the basic combination as a type definition and define the other valid combinations as type *Variations*.

4.1. Types of "standardization RA"

In this section, we take the case where the goal of a reference architecture is *standardization* (i.e., we fix the value in the G1 sub-dimension to "standardization"). As already noted in the early ages of software architecting, an attempt to standardize architectures at an early stage is doomed to fail [40]. In other words, standardization reference architectures should not incorporate innovative (untested in practice, or still not accepted by practitioners) elements. This precludes the possibility of combining the *preliminary* and *standardization* values from the multi-dimensional space. Thus, standardization reference architectures should be *classical* reference architectures (fixing the value in the C3 sub-dimension to "classical"). Next, we sequentially fix the two possible values for the C1 sub-dimension and discuss the effect of this selection on the possible values for the other sub-dimensions. We start by fixing the values in the C1 sub-dimension because it has only two, mutually exclusive values, which simplifies the application of the backtracking algorithm (in contrast to the C2 sub-dimension, which has four, mutually inclusive values).

Type 1: Reference architectures of Type 1 are *classical*, *standardization* architectures designed to be implemented in *multiple organizations* (see Table 1). Representative sets of *user* and *software* organizations should be involved in the architecture definition as standardization requires a consensus in order to be successful. The design effort is conducted by a *standardization organization* (roles denoted with "D" and "R" in Table 1, as discussed in Section 3.1). Architects from the software organizations may be invited to act as "designers" in the standardization organization. The involvement of a standardization organization facilitates the attainment of consensus and the establishment of the architecture as a standard. This organization may be an officially established consortium of software and user organizations or an independent standardization organizations (commercial, governmental or non-governmental). Its role is to involve the user and software organizations that have relevant market presence and moderate the architecture definition process, unifying the (potentially) conflicting views the stakeholders. Note that research organizations do not have a crucial role in a standardization effort. The classical nature of Type 1 reference architectures implies usage of established results in practice. The involvement of research organizations may be valuable only in providing survey and summarization data on the best practices from the field from a research perspective. As their role is not crucial, we do not list them separately in Table 1 (if present they operate under the umbrella of the standardization organization). Reference architectures of Type 1 contain a description of the *components* and *interfaces* as these are the elements that are the target for standardization [40]. General *guidelines* and *policies* that define the main line for the architecture implementation accompany them. Note that specification of other elements than these is not excluded, but will not contribute to achieving the main goal of an architecture and may even have a negative effect on it (by providing unnecessary information and decreasing the efficient adoption of the architecture). Components are defined at a *high level of aggregation* as standardization of internal component details is unnecessary. Interfaces are defined at more detailed levels as their precise specification is crucial for achieving interoperability. Type 1 reference architectures are *abstract* (i.e., its elements are defined at a high level of abstraction) as each organization should have the freedom to select the concrete implementation details according to its own settings. Type 1 architectures are *semi-formal* in order to provide a clear standard specification and to allow stakeholders who typically are inexperienced in strong formalization techniques to understand them. Example reference architectures of Type 1 are the WRM [24], CORBA [25] (both discussed in Section 6.2), the OSI RM [26], and OATH [41].

Variant 1.1: A standardization effort led by one software organization will face resistance from its market competitors and will not be able to establish a reference architecture as a domain standard. However, if the standardization goal is limited to its client base, a software organizations may define a standardization reference architecture. Its incentive may be, for example, to use a standardization reference architecture for ensuring interoperability between the systems designed and developed at its clients. Thus, Variant 1.1 covers the case when a *software organization* defines a *classical*, *standardization* reference architecture that will be applied in *multiple organizations* from the client base of the software organization. As the architecture will reflect the vision of a single organization, in contrast to Type 1, the architecture elements may be also *concrete* and *semi-concrete* (leading to a variation in the D3 sub-dimension) and have *detailed* or *semi-detailed* components and policies/guidelines (leading to a variation in the D2 sub-dimension). The variations in the D2 and D3 sub-dimensions are a result of the "monopolistic" position of the software organization in this scenario. The software organization may like to ensure the usage of preferred software products (e.g., because they are owned or marketed by the company or because they are compatible software solutions). Hence, a more concrete architecture will suit its goals in this context. Similarly, the variation in the D2 sub-dimension reflects the possible preference for more elaborate guidelines, and components decomposition ensuring proper usage of the standard in all projects. Software product line architectures ([18,21,22]) that are applied for the design of customizable software solutions for clients fall in this variant of reference architectures. As already discussed software product line architectures may have a formal specification (leading to a variation in the

Table 1
Type 1 reference architectures.

Dimension	Values
G1: Why	Standardization
↓	↓
C1: Where	Multiple organizations
C2: Who	Standardization organization (D), Software organizations (R), User organizations (R)
C3: When	Classical
↓	↓
D1: What	Components, interfaces, policies/guidelines
D2: How	Aggregated components and policies/guidelines; (semi) Detailed interfaces
D3: How	Abstract elements
D4: How	Semi-formal element specifications

D4 sub-dimension). An example for Variant 1.1 reference architecture is the PinkRoccade Local Government Reference Architecture (PRLG RA) [42].

Variant 1.2: A standardization effort in a domain conducted solely by one or set of *user organizations* would face lack of capabilities and experience (invitation of software organization will naturally lead to the formation of a consortium and a Type 1 reference architecture). However, a user organization may define a reference architecture for itself aiming at standardization of certain aspects of software delivered to it from external software developers (e.g., components, interfaces for components, mechanisms used to connect components [16]). Usually, to compensate for the lack of capabilities and experience, the reference architecture design will be done by a temporary employed entity (e.g., as a consultancy service). The reference architecture is used to ensure satisfaction of organizational architectural requirements when numerous external software organizations compete for the development of certain components for the user organization. Thus, in this variant a *user organization* defines a *classical*, *standardization* reference architecture that will be applied for the design of concrete architectures in *multiple organizations* that provide software solutions to the user organization. In this variation, designers from the external software organizations should still be involved in the design process. Their involvement ensures that the architecture will reflect their experiences and capabilities for software design. This variant introduces a variation in the D3 sub-dimension only. Similar to Variant 1.1, the local character of the reference architecture allows the user organization to elaborate *semi-concrete* or *concrete* reference architectures, making certain choices in terms of software solutions, technology, etc. In contrast to Variant 1.1, this variant does not introduce a variation in the D2 sub-dimension (as only high-level components and their interfaces will be valuable for standardization). Examples for this variation of Type 1 reference architectures are the DoD RA [16] and the GMSEC RA [43].

Type 2: Reference architectures of Type 2 are *classical*, *standardization* architectures designed to be implemented in a *single organization* (see Table 2). These reference architectures are designed to serve as a standardization tool for the design of a set of software solutions within the organization. Software *user groups* (from the organizational units where the software is applied) provide requirements that designers from the *software design groups* (from the organizational units where the software is applied) use in the architecture design. Note that the designers may be working on a temporary basis for the organization, e.g., on a consultancy project basis. Designers may also provide architecture requirements. Managers from the organization should ensure application of the standard in the company projects. Analogously to Type 1 architectures, reference architectures of Type 2 define architectural *components, interfaces* and *policies/guidelines* are *semi-formal* or *formal* (in the case of some software product line architectures). The elements of reference architectures of Type 2 can be defined at *any level of aggregation* depending on the specific organization context. These architectures make certain choices in terms of technology, applications, and standards as a consequence of their standardization goals within the concrete organization. That is why they are *concrete* or *semi-concrete* reference architectures. An example Type 2 reference architecture is the Fortis Bank Reference Software Architecture[3] (FORTIS RSA – a reference architecture used in the software development of a specific European bank). Software product line architectures that are defined to ensure efficient and effective design and development of software used in the organization are also of Type 2 (in this context, the stakeholders are also referred to as "domain engineering unit" [14]).

Table 2
Type 2 reference architectures.

Dimension	Values
G1: Why	Standardization
↓	↓
C1: Where	Single organization
C2: Who	Software design groups (D, R), User groups (R)
C3: When	Classical
↓	↓
D1: What	Components, interfaces, policies/guidelines
D2: How	Aggregated, semi-detailed, or detailed elements
D3: How	Semi-concrete or concrete elements
D4: How	Semi-formal or formal element specifications

The users of the software solutions based on a reference architecture may be outside the organization borders. For example, software produced through the software product line approach is often used by users external to the organization that produces the software. This variation, however, has no effects on the design sub-dimensions. Therefore, we do not discuss it as a separate variation. Note that an analogous observation can be made for Type 4 reference architectures.

4.2. Types of "facilitation RA"

In this section, we discuss the case where the goal of a reference architecture is *facilitation*. We sequentially "fix" the two possible values for the C3 and C1 sub-dimensions and discuss the effect of this selection on the possible values for the rest of the sub-dimensions. Analogous to our approach in Section 4.1, we start with the C3 and C1 sub-dimensions (as each of them has two, mutually-exclusive values). We start with the "*classical*" value of the C3 sub-dimension and the "*multiple organizations*" value of the C1 sub-dimension.

Designing a classical reference architecture that can facilitate the design at multiple organizations by multiple *software organizations* is not sensible. Such a scenario would imply that software organizations (operating and competing in a domain) might lose their advantage over their competitors without obtaining any benefits from this effort. *User organizations* will not have an incentive of designing a reference architecture that facilitates their software providers as well. Therefore, only two possibilities for the design of a facilitation architecture to be applied at multiple organizations exist, i.e., a reference architecture to be designed by an independent organization or by a single software organization (which can be seen as analogous to Type 1 and Variant 1.1 in the context of facilitation reference architectures). Next, we discuss these two cases.

Type 3: We call a *classical*, *facilitation* architecture designed for *multiple organizations* by an *independent organization* a Type 3 reference architecture (see Table 3). The independent organization may be one or more research centers, a non-profit organization, governmental or non-governmental organization. An independent organization can have the capabilities and resources (time, people) for defining a classical, facilitation reference architecture. Furthermore, it has the freedom to define the "best" reference architecture based on existing experiences in research and industry not being influenced by any political, strategic, or legacy issues and introduce a positive impact on the domain. The design of a reference architecture from this type should be based on requirements from *software* and *user organizations*. Their active involvement is critical as it ensures proper reflection of their requirements and facilitates dissemination of the reference architecture after its design. Inability to involve them indicates lack of interest in the industry for

[3] Due to their proprietary character, several of the architectures discussed in this paper are not published.

Table 3
Type 3 reference architectures.

Dimension	Values
G1: Why	Facilitation
↓	↓
C1: Where	Multiple organizations
C2: Who	Independent organization (D), Software organizations (R), User organizations (R)
C3: When	Classical
↓	↓
D1: What	Components, interfaces, policies/guidelines
D2: How	Semi-detailed components and policies/guidelines, Aggregated or semi-detailed interfaces
D3: How	Abstract or semi-concrete elements
D4: How	Semi-formal element specifications

Table 4
Type 4 reference architectures.

Dimension	Values
G1: Why	Facilitation
↓	↓
C1: Where	Single organization
C2: Who	Software design groups (D, R), User groups (R)
C3: When	Classical
↓	↓
D1: What	Components, policies/guidelines
D2: How	Aggregated or semi-detailed components Semi-detailed or detailed guidelines
D3: How	Abstract, semi-concrete, or concrete elements
D4: How	Semi-formal or informal element specifications

such a reference architecture and is a prelude for its oblivion. A point of attention in this type is the possibility for inclusion of "preliminary" elements in the reference architecture. Inclusion of preliminary elements in the architecture will be natural for a research organization. However, it will be in violation with the classical nature of the architecture. Such inclusions will lead to distrust in the potential users of the architecture and eventually to the weak usage of the architecture.

Type 3 reference architectures should define the main *components, interfaces, guidelines and policies*. Components and policies and guidelines should be *semi-detailed*. Too detailed specification of components will make the application harder to apply in the various contexts of the organizations. Too aggregated specification might be seen as adding too little value for the domain and the architecture will not function as a facilitation tool. The policies and guidelines should also be *semi-detailed* in order to facilitate understanding and implementation of the architecture by practitioners. Interfaces should be aggregated or semi-detailed as the goal is only to specify their main aspects rather than standardize them. As the architecture is targeted towards multiple organizations, it should abstract as much as possible from context specific details. However, its classical nature and facilitation goal mean that certain technological choices can be incorporated in it (facilitating the choice of the technology that performs best at present, gives best opportunities, etc.). Similar to Type 1 architectures, Type 3 architectures are *semi-formal* as they should be easily understood but still provide a clear specification of the architecture. Examples for Type 3 reference architectures are Mercurius RA [44] (a RA for workflow management systems), RA for Web Browser [45], RA for Web Servers [46], INAHL [47] (a reference software architecture for the development of applications in the petroleum industry).

Variant 3.1: Analogously to Variant 1.1, a *software organization* may have the capabilities and resources to design a *classical*, *facilitating* reference architecture for *multiple organizations*. The incentive for a software organization may be to market its software products through the reference architecture, ensure quality of architecture designs among its client base, shorten development times, and sell the architecture to other market participants. The software organization would have a direct contact with its client base and would be able to collect requirements and disseminate the architecture among them. Similar to Variant 1.1, Variant 3.1 varies in the D3 and D2 sub-dimension from its type definition. In Variant 3.1, the reference architecture can be also *concrete* (as the software organization may aim at using its own software products). The local character of the reference architecture also allows the architecture specification in more details (hence, *detailed* components and policies also fit in this variant). Example reference architectures of Variant 3.1 are the Microsoft Application Architecture for .Net [48], IBM PanDOORA [49], and CORA [50].

Type 4: Reference architectures of Type 4 are *classical*, *facilitation* architectures designed to be implemented in a *single organization* (see Table 4.). They are similar to Type 2 architectures but are designed only as a facilitation (guidance) tool in the design and implementation of systems in the organization. Due to their similarity to Type 2 architectures they need a similar stakeholder representation. In this case, however, the enforcement power of managers is less important as the architecture does not have to function as a standard. The facilitation role of Type 4 architectures requires their representation to be *semi-formal* or even *informal*. An *aggregated* or *semi-detailed* component design suffices for achieving the architecture facilitation goal. As they are designed for a concrete organization, they can be technology independent (*abstract*) or indicate technology choices (*semi-concrete*, *concrete*) Examples of software reference architectures of Type 4 are the Achmea Software RA [51] (a reference architecture for software development in the largest Dutch insurance group) and the ABN-AMRO Web Application Architecture [52].

As a next step, we investigate the "preliminary" value of the C3 sub-dimension for facilitation architectures. Design of a *preliminary*, *facilitating* reference architectures for a *single organization* is plausible. However, the effort of defining a preliminary reference architecture for a single organization requires substantial resources. Only leading organizations might invest in visionary architectures. As we did not find examples for such reference architectures, we do not define it as a separate type. Next, we discuss the "*preliminary*" value of the C3 sub-dimension for *multiple organizations*.

Type 5: Reference architectures of Type 5 are *preliminary*, *facilitation* architectures designed to be implemented in *multiple organizations* (see Table 5.). They are designed to facilitate the design of architectures of systems that will become needed in the future. As these architectures are preliminary, a *research center* (or a group of research centers) together with interested, innovatively oriented software organizations are elaborating the architecture design. In order to address the *user* and *software* design requirements, organizations representing these roles should be involved in the design process. These reference architectures are innovative in their nature and have to define the *components* required in a system implementing it, *algorithms* that can be used to support the operation of the components, and *protocols* that demonstrate the interactions among the components. The components and algorithms have to be *detailed* or at least *semi-detailed* as they have to provide details for the innovative aspects they define (showing implementability of components, clarifying their operation, etc.). Protocols are used only to demonstrate the general lines of communication taking place. Hence, they are specified at a more aggregated level. Reference architectures of Type 5 abstract from a concrete technology, as it will not exist or will be rather immature at the time of architecture design (hence, they are *abstract* architectures). Unambiguousness and evidences for their qualities are important to convince

Table 5
Type 5 reference architectures.

Dimension	Values
G1: Why	Facilitation
↓	↓
C1: Where	Multiple organizations
C2: Who	Research centers (D), Software design organizations (D, R), User organizations (R)
C3: When	Preliminary
↓	↓
D1: What	Components, algorithms, protocols
D2: How	Detailed or semi-detailed components and algorithms Aggregated or semi-detailed protocols
D3: How	Abstract elements
D4: How	Formal or semi-formal element specifications

the usage of the architecture for the design of the first systems from this class. That is why they are *formal* or *semi-formal* architectures. The NCSTRL architecture [53] (a reference architecture for federated digital libraries) is an example of Type 5 reference architecture. The ANSI-SPARC database reference architecture [40] is an example of a reference architecture that closely resembles Type 5 (discussed in Section 6.2).

Variant 5.1: In this variant, a *preliminary*, *facilitating* reference architecture designed for *multiple organizations* is designed only by a *research center*. The origin in pure research environment of these reference architectures results in "futuristic" designs that do not concentrate on the requirements of the domain stakeholders but on the innovative elements of the architecture. That is why these architectures are usually not considered for system implementations in practice. Their main contribution is in inspiring future research efforts in the domain (depicting main software issues, providing blueprints for prototype implementations, etc.). The success of these architectures can be estimated by their impact on the research community (e.g., a citation number). Examples for this variant are ERA [54] (a reference architecture for development of e-contracting systems), AHA [55] (a reference architecture for development of adaptive hypermedia systems), and eSRA [56] (a reference architecture for the development of systems supporting sourcing solutions).

5. Usage of the framework

The framework for reference architectures presented in the previous sections can be used in three ways. First, it can be used for the analysis of existing reference architectures. Second, it can be used at architecture design-time to "safeguard" the design of congruent reference architectures. Third, it can be used when an existing reference architecture has to be re-designed due to changes in the environment (i.e., it has to evolve), indicating the directions for architecture updates. Next, we discuss the application of the framework in these three scenarios.

5.1. Application of the framework for the analysis of reference architectures

In order to analyze an existing reference architecture for its congruence, it is first positioned in the multi-dimensional space presented in Section 3 (see Fig. 3). When the values of the reference architecture are identified, they are mapped to the five architectural types and their variants. The type in which these values match best (fully or partially) is selected as a basis for the analysis step. An architecture may match in its goal value with one type and in its context values with another type. In this case, the architecture is classified as both candidate types and the analysis step is performed against each of the types (as indicated in Fig. 3). Note that for architectures that have evolved in time (see Section 5.3) the analysis process has to be performed for each of the evolution phases of an architecture (this is the case for example for the GMSEC RA [43] that has evolved in time in its goals or contexts, see Table 6).

The analysis performed with the framework is purely qualitative. The type descriptions (presented in Section 4) are the basis for the analytical work performed in this step. If an architecture matches fully with a type, the architecture is congruent and the analysis is completed. If there are architecture values that do not match with the type values, the architecture is incongruent and the effects of the divergences with its type are scrutinized.

The analysis step is rather complex in its nature because of the context specificity of the "weight" of the divergence effect (i.e., how serious is the impact of the divergence on the architecture success). A divergence in one context may be fatal for the architecture adoption while in another to have less damaging impact. For example, divergence in the level of formalization (D4) by elaborating a formal architecture where a semi-formal one is recommended may have lower impact if there is some formal modeling culture in the intended audience. Furthermore, depending on the context, the weights of divergences differ per dimension. For example, for a Type 5 architecture, divergence in the level of formalization (D4) may have a lower impact than divergence in the architectural elements (D1), which are crucial for the architecture success. This context specificity of the "weight" of the divergence effect impeded us in the elaboration of a precise method for the analysis of incongruent architectures.

5.2. Application of the framework for the design of reference architectures

As a first step in the application of the framework for the design of a reference architecture, the sponsor of the reference architecture explicitly states the architecture goal, the architecture application context (single or multiple organizations) and the timing aspects of the architecture (classical or preliminary). This step ensures the explicit positioning of a reference architecture in its context and the explicit articulation of its goals. The omission of this step (or its implicit performance) is a premise for different interpretations by the stakeholders of the fundamental aspects of a reference architecture and for poor design choices.

Next, the sponsor matches the values in these dimension values to the architecture types and variants defined in the framework (see Fig. 4). If no match is found (and hence the intended reference architecture is not congruent), the application context, goals and timing aspects have to be revisited using the architecture types (and variants) as a guidance tool. The goal may be changed or the scope may be redefined to try to arrive at a congruent reference architecture. The lack of a match may be an indication for poor initial choices that can be resolved by elucidating the architecture dimension values but may be an indication for a serious, hidden conflict in the intentions of the sponsor and the application/design contexts (e.g., as discussed in Section 6, innovative architectures are often conceived with standardization goals). Revelation of a conflict that cannot be resolved should lead to the discontinuation of the project at an early stage saving the investment of valuable time and efforts in an initiative facing otherwise a failure.

If a match is found, the sponsor approaches the stakeholders that have to be involved according to the type/variant description. If the sponsor cannot involve the stakeholders in their roles, again, the context and goals have to be revisited. This is a process in which a match is sought between the goals and possible contexts in the concrete case and those defined in a type. Typically, such a match should be reached in at most two iterations of this process,

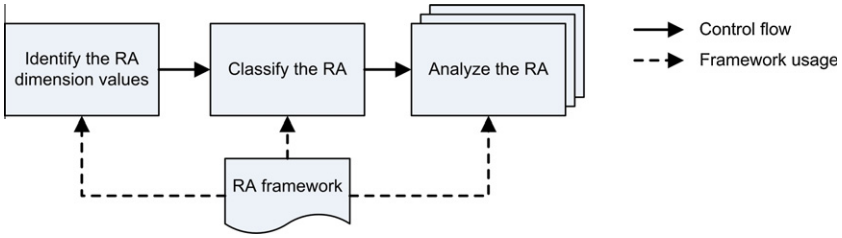

Fig. 3. Framework usage in the analysis of a reference architecture.

Table 6
List of analyzed reference architectures. "T/V" denotes the architecture type/variant. "X" denotes a match of the architecture values with those in the type/variant, "≈" that deviations occur, and "–" that there is no match.

		T/V	G1	C1	C2	C3	D1	D2	D3	D4
1	WRM [24]	1	X	X	X	X	X	X	X	X
2	OSI RM [26]	1	X	X	X	X	X	X	X	X
3	CORBA [25]	1	X	X	≈	≈	X	X	X	X
4	HLA [57]	1	X	X	≈	≈	X	X	X	≈
5	ANSI-SPARC [40]	1(5)	X(-)	X(≈)	X(X)	-(X)	X(≈)	X(≈)	X(X)	X(X)
6	HISA [58]	1	X	X	X	X	X	X	X	X
7	OATH [41]	1	X	X	X	≈	≈	X	X	X
8	AUTOSAR [59]	1	X	X	X	X	X	X	X	X
9	PRLG RA [42]	1.1	X	X	X	X	≈	≈	X	X
10	GMSEC RA [43]	2	X	X	X	X	X	X	X	X
		(1.2) (1)	(X)(X)	(–)(–)	(≈)(≈)	(X)(X)	(X)(X)	(X)(X)	(X)(≈)	(X)(X)
11	FORTIS RSA	2	X	X	X	X	X	X	X	X
12	Mercurius RA [44]	3	X	X	≈	≈	≈	≈	X	X
13	RA WS [46]	3	X	X	≈	X	≈	X	X	X
14	INAHL [47]	3	X	X	≈	X	≈	X	X	X
15	RA WB [45]	3	X	X	≈	X	≈	X	X	X
16	MS .NET [48]	3.1	X	X	X	X	X	X	X	X
17	PanDOORA [49]	3.1	X	X	X	X	X	X	X	X
18	CORA [50]	3.1	X	X	X	X	X	X	X	X
19	ACHMEA RA [51]	4	X	X	X	X	X	X	X	X
20	ABN WAA [52]	4	X	X	≈	≈	X	X	X	X
21	NCSTRL [53]	5	X	X	X	X	X	X	X	X
22	ERA [54]	5.1	X	X	X	X	≈	X	X	≈
23	eSRA [56]	5.1	X	X	X	X	≈	X	X	≈
24	AHA [55]	5.1	X	X	X	X	X	≈	X	X

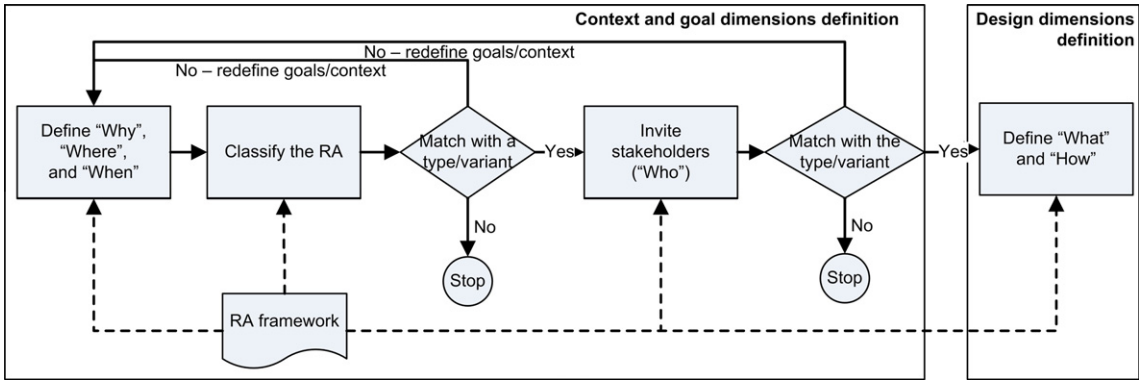

Fig. 4. Framework usage in the design of a reference architecture.

as in the second iteration the parties that can be involved would be known and the possible choices for the other sub-dimensions would be evident. Again, this decision point may provide indications for a substantial problem in the sponsor intentions. For example, a desire to define a standardization architecture where major stakeholders cannot be involved will be detected at this stage. If no match can be reached, the sponsors may stop the architecture design process. If a match is reached, the framework is used to identify the design guidelines.

Software product line architectures have relatively well-defined goals (both at a strategic and operational level) and context [18]. Moreover, there is an awareness about them in the architecture community. Practically, this means that in the case of software product line architectures the sponsors have already been given elaborate tools to support them in the "context and goal definition" phase. However, such "tooling" for the other types of reference architectures is currently lacking. Our work provides the basis for such a support.

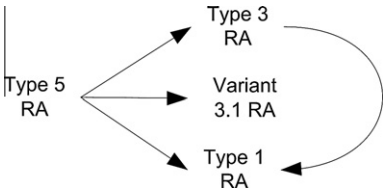

Fig. 5. An example evolution path for Type 5 reference architectures.

5.3. Application of the framework for the evolution of reference architectures

A reference architecture may evolve from one type to another influenced by changes in the environment (technology developments, accumulated experiences, etc.) or changes in its goal. To stay usable, a reference architectures should be adapted to reflect these changes. For example, a preliminary reference architecture will have to be transformed into a classical architecture when the domain matures. Similarly, transformation from a facilitation effort to a standardization effort or from a "single organization" scope to "multiple organization" scope may be required.

In Fig. 5, we show examples of evolution paths of a reference architecture of Type 5. The evolution in this example is caused by changes in the domain maturity. When experience in a domain is accumulated, a Type 5 reference architecture may evolve into a Type 3 (or its variant 3.1) facilitating architecture designs in the domain based on a recognized best practice. At a later stage, if standardization becomes essential for the domain, a Type 3 reference architecture may evolve into a Type 1 reference architecture (note that Variant 3.1 reference architecture can hardly evolve into Type 1 as it reflects the vision of a single organization). Alternatively, if standardization is recognized as vital for the domain from the very beginning, a Type 5 reference architecture may directly evolve into a Type 1 architecture. Although not depicted in Fig. 5, a Type 5 architecture may also evolve into a Type 2 or Type 4 architecture if an organization decides to adopt it as an internal reference architecture. Concrete examples for the evolution of reference architectures are GMSEC RA [43] (evolved from Type 2 to Type 1.2 and eventually to Type 1) and CORA [50] (currently under discussion to evolve from Type 3.1 to Type 3). However, in neither of these two examples, actual adaptation on the architectures have taken place – only the new architecture goal and context were identified.

Our framework provides a convenient guiding tool for the transformation of a reference architecture from one type into another type. The main steps in such a transformation process are depicted in Fig. 6. The process is mainly a combination of the activities presented in Fig. 3 and Fig. 4. As a first step, the values of the dimensions for the existing reference architecture have to be identified. In step 2, the type of the "architecture-to-be" has to be identified. In essence, this step is analogous to the application of the framework for the design of a reference architecture. As a last step, the changes that have to be made to the old architecture in its evolution to the architecture-to-be are identified. These changes follow from the differences in the values in the design sub-dimensions of the old and "to-be" architectures.

6. Framework validation

In our validation of the framework, we have focused on its application as an analytical tool. Demonstrating its value in this scenario shows correctness of its structure and elements which are crucial for the application of the framework in the design and re-design scenarios (as can be also observed from the activities depicted in Fig. 3, Fig. 4, and Fig. 6). To validate our framework for its applicability and its quality as an analytical tool, we have applied it for the analysis of 24 reference architectures. The criteria for the selection of the 24 reference architectures were variety (representing the different types and variants of reference architectures in our framework), availability of documented resources on them, and responsiveness of architecture creators and experts to our requests for clarifications. In this section, we first analyze the application of the framework in these 24 cases. Next, we validate the correctness and relevance of the conclusions drawn during the framework application. The section ends with a discussion. We note that as a result of the validation process two minor improvements have been introduced to the framework (discussed in Section 6.1). The framework presented in the paper incorporates these improvements.

6.1. Framework application

The list of reference architectures analyzed and their classification according to our framework are provided in Table 6. A row in the table shows the types/variants that the reference architecture resembles the most and its values for each of the sub-dimensions

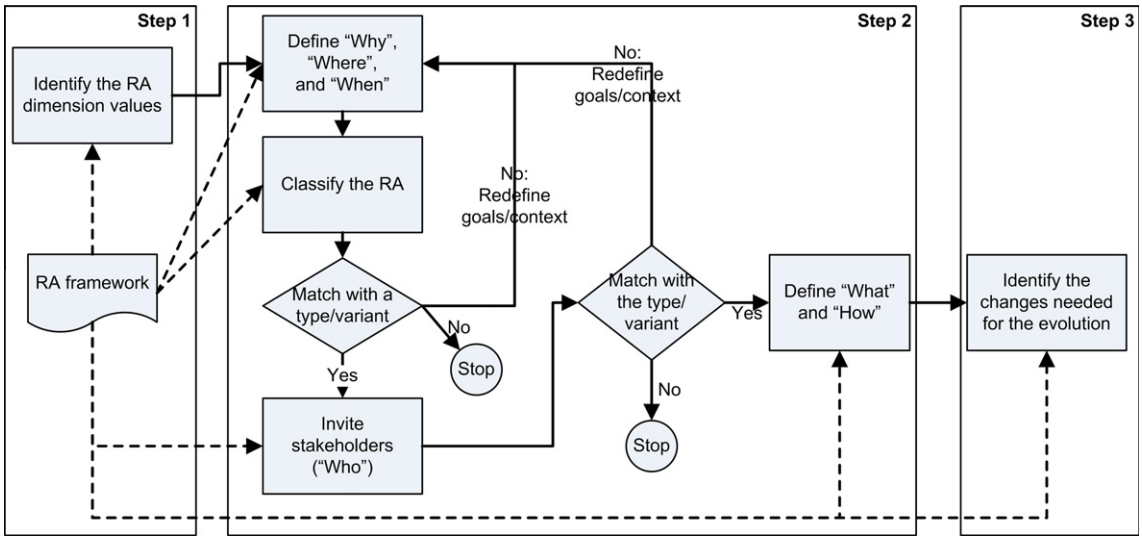

Fig. 6. Framework usage in the evolution of reference architectures.

in our framework. The role of the gray color is explained in Section 6.2.

Next, we discuss our main observations and conclusions on the applicability of the framework as an analytical tool. We address consecutively each of the three steps of the analysis process presented in Fig. 3.

Conclusions on the dimensions identification step: The "dimension-values" of the reference architectures studied were obtained from existing publications on these architectures and from personal communications with experts involved in their design. The identification of the dimension-values was straightforward for the design sub-dimensions (D1, D2, D3, D4). In few of our cases, the architecture goal (G1) was only implicitly stated in the architecture documentation (e.g., [56,55]). In all cases that we analyzed, the identification of the context values was straightforward for the "Where" and "Who" sub-dimensions (C1, C2). However, for the "When" sub-dimension (C3), we noted that an architecture may be preliminary even when the context provides sufficient experience in the domain. The reason is that designers aiming at providing the "best" architecture easily deviate from the tested approaches and propose new, little or not tested designs (e.g., [41,25,40]). Thus, the value for this sub-dimension is defined either by the context or by the choice for the context made by the designers. As a result of this observation, we have adapted the description of our classification space to reflect this nuance.

Conclusions on the architecture classification step: Classification of an architecture within a type was straightforward when both goal and contexts values (partially) matched with the values of one type. However, when there was a substantial mismatch in their values, the choice of a type in which the architecture fits was not evident. We faced this problem in the case with the classification of the ANSI-SPARC DBMS reference architecture [40]. The ANSI-SPARC DBMS reference architecture was conceived as a standardization effort suggesting a Type 1 architecture. Its values for the design sub-dimension were also compliant with those prescribed in Type 1. However, the architecture was preliminary. As its designers concluded "post-factum", the existing technology was not able to support it and standardization was not an appropriate goal [40]. In this example, we analyzed the architecture both from the perspective of Type 1 as well as Type 5, investigating the consequences for the architecture usage caused by the deviations within each of the types. Consequently, we have adapted our analysis process to allow analysis of an architecture from the perspective of multiple architectural types.

Conclusions on the architecture analysis step: Based on the conclusions from the architecture classification step and the type descriptions provided in our framework, we analyzed the factors that could possibly cause lower or more difficult adoption of the reference architectures studied. The analysis step was a reasoning exercise supported by the descriptions postulated in the architecture types. The postulates provided us with the arguments for the selection of a certain value in a type. We used these arguments for drawing conclusions about the consequences for an architecture in the cases of value mismatches. The quality of our conclusions is discussed next.

6.2. Quality of analysis results

To evaluate the validity of the conclusions drawn with the help of our framework, we compared the conclusions from our analysis with the successes and shortcomings of the reference architectures revealed in their practical application. Obviously, the successes and shortcomings of a reference architecture can be measured only from the perspective of time. Thus, we have only investigated the correlation between our conclusions and the records for the longer-existing reference architecture (more than 5 years). These architectures are shown in gray in Table 6. In all cases, we found indications for a connection between deviations of a reference architecture from its type and the shortcomings of the reference architecture during its practical application. Next, we provide evidences supporting this statement. These evidences were found in scientific and industry records on the usage of the architectures, reviews made by experts, as well as opinions of people involved in their design and application.

CORBA: CORBA was designed as a standardization architecture aimed at multiple organizations [25] (hence a Type 1 architecture). However, it was a preliminary architecture: "the specification was large and complex and much of it had never been implemented, not even as a proof of concept" [60]. The inclusion of preliminary architectural elements (related to the deviation in C3) is seen as one of the main reasons for the limited success of CORBA. In addition, the lack of involvement of *software organizations* – "no commercial CORBA vendor made a commitment to implement CCM" [60] (related to the deviation in C2) is seen as a contributing factor for the limited success of CORBA [60].

HLA: The High Level Architecture for simulations [57] was designed to standardize the development of simulation systems for the USA military services, fostering interoperability and reusability of simulation applications. The architecture was accepted by the community and became a standard for the USA and NATO military simulation systems [57,61]. Although significant experience with design of military simulation systems has been accumulated prior to its design [57], the idea for interoperability of simulations was relatively novel and no substantial experience on it was collected. The involvement of research institutes (deviation in D2) in its design also naturally led to the novel nature of the architecture (deviating in C3 of Type 1). As a result, the HLA had preliminary elements for which the community needed clarifications [62]. Furthermore, some elements were specified in an informal way (deviating in D4). The informally specified elements left space for different interpretations which required the provision of additional interpretation documents [61].

ANSI-SPARC: As already discussed in Section 6.1, the ANSI-SPARC DBMS architecture is a Type 1 architecture [40]. However, its design team concluded that "no existing or proposed implementation of a database management system completely satisfies" the requirements identified by the design team "nor comprises all of the concepts involved" [63] (conflicting with the Type 1 value in C3). The initially designed ANSI-SPARC DBMS standardization reference architecture required the elaboration of additional architectural elements typical for Type 5 reference architectures. Thus, the architecture initially designed as Type 1 had to be transformed into a Type 5. This was done in the years after the initial design took place and contributed for the wide acceptance nowadays of the three-level database model [64]. The original architecture itself never became well-known and its standardization-specific architectural elements had little effect on the domain.

Mercurius RA: The Mercurius Reference Architecture for workflow management systems [44] is a facilitation architecture designed for multiple organizations. However, it was designed mainly in a research center (deviating in C2), had a preliminary architectural element (deviating in C3), was too detailed at certain points and missed description of application guidelines and architecture interfaces (deviating in D1 and D2). Thus, it deviated in a number of dimensions from a Type 3 architecture. As concluded by one of its designers the origin of the architecture predominantly in a research environment led to these deviations and eventually to the lack of attention by the domain to the Mercurius RA. To the best of our knowledge, it was never used as a basis for the design of a leading concrete architecture.

RA WS: The Reference Architecture for Web Servers [46] is a classical reference architecture aiming to facilitate the design of

new web servers (Type 3 architecture). The architecture was designed in pure academic environment on the basis of three open-source web server architectures (deviating in C2). The architecture specifies the required components but does not address the interface and guidelines architectural elements (deviating in D1). As acknowledged by one of its designers, the RA WS was reported to be used only as an introduction material to new employees at the Apache group. Its weak usage can be linked to the lack of industry involvement in the design effort (related to the deviation in C2).

ABN WAA [52]: The ABN-AMRO Web Application Architectures was designed to facilitate the development of web applications – a novel solution by the time of the architecture design. However, as reported by one of its designers, its impact in the organization was very limited. This was mostly due to insufficient leadership that had to ensure usage of the architecture (designed in one department but designed for other departments that were the actual users of the architecture) and the involvement of only a limited number of stakeholders (related to the deviation observed by us in C2). Furthermore, the novel nature of web application development resulted in the design of an architecture that was not built on proven practices. Instead, it included a number of innovative ideas that had to be proven in practice (related to the deviation in C3). That also obstructed the adoption of the architecture.

AHA: The Reference Architecture for Adaptive Hypermedia Applications is a preliminary, facilitation reference architecture designed solely in a research center [55]. By the time of writing of this article, the architecture has been referenced in 80 publications. The architecture components are described at a high level of aggregation, deviating in D2 of Variant 5.1. The lack of sufficient details in the architecture is criticized in [65].

The *WRM* [24], *OSI RM* [26], *MS .NET* [48], *IBM PanDOORA* [49], and *NCSTRL* [53] architectures fully match their respective types. These reference architectures are also reported to have been adopted and applied in numerous projects [66–70].

6.3. Discussion

Based on the observations in Sections 6.1 and 6.2, we conclude that our framework can be applied for the analysis of reference architectures. It facilitates drawing of valid conclusions on the congruence of reference architectures and on potential pitfalls for their wider adoption.

After the application of the framework on 24 cases, we observed two main trends of deviations from our architectural types. The first trend concerns Type 1 architectures and has to do with the inclusion of preliminary elements in a standardization architecture. As noted in [60]: "there is no room for innovation in standards. Throwing in 'just that extra little feature' inevitably causes unforeseen technical problems, despite the best intentions". The inclusion of preliminary elements leads to the implicit mixture of Type 1 and Type 5 reference architectures. However, this mixture is usually not reflected in the architecture design and Type 5 elements (algorithms, implementation proofs) are not included in the architecture. We observed this deviation in its extreme form in the case of the ANSI-SPARC DBMS reference architecture [40] and in milder form in CORBA [25] and OATH [41]. In essence, our framework explicitly positions preliminary reference architectures as a separate type showing the way they can deliver maximum value but also indicating how their design, goal, and contextual congruence may be lost. The lack of attention for the role and value of preliminary reference architectures in the architecture community has led to numerous examples of incongruent reference architectures. Thus, our framework shows the limited nature of preliminary reference architectures in the design space of all reference architectures (addressed only in a single architectural type) and at the same time points out their importance and ways to properly position them in terms of goals, context and design choices.

The second trend we observed in Type 3 architectures. Researchers are often inspired to propose a reference architecture for a domain based on existing best-practices (see, e.g., INAHL [47], Mercurius RA [44], RA WS [46], RA WB [45]). However, they often develop their architectures in isolation from the industry. This makes it hard to disseminate the resulting architecture and convince about its quality. Although these are potentially good designs, they have low adoption rates in practice. In addition, the natural strive for innovation of researchers easily misleads them in adding novel, untested elements, which contradicts the classical nature of Type 3 architectures.

7. Related work

The literature on reference architectures is scarce. Definitions and brief explanations on reference architectures are provided in [3] and [71]. In [7,8], the authors acknowledge the lack of clear understanding of reference architectures and provide a multi-dimensional classification of reference architectures. The classification is, however, not based on formal research. In [2], an overview of reference architectures is presented. The authors concentrate on the description of the many facets of reference architectures. The goal of the paper is to "create guidelines for the content of a reference architecture and the process to create and maintain it". The dimensions discussed in [2] are less explicit compared to our work and have a descriptive goal. The dimensions discussed in our paper are clearly structured and are defined to serve as a framework for the classification of the level of congruence of goals, context, and design of reference architectures. In [2], the authors acknowledge the need for congruence of different dimensions for the definition of successful reference architectures but do not state requirements with respect to these dimensions.

Software product line architectures have been in the focus of academia and industry due to the relatively clear, measurable benefits introduced by them and well-defined design and application scopes. The popularity of software product line architectures concept in academia and industry has lead to numerous publications that aim at clearly defining the business goals pursued with the introduction of product line architecture [18,22,72]. For example, the design of software product line architectures with congruent business goals, functionalities and qualities is addressed in [17,19]. The role of specific activities on the success of a product line is discussed in [73]. These efforts have created high awareness in the domain on the needs for reaching a congruent software product line architecture and consequently have improved the design of congruent product line architectures. With respect to our framework, however, these efforts are limited to only subsets of the reference architectures in Variant 1.1 and Type 2 architectures (see Section 4.1).

As already mentioned in Section 1, the quality of a reference architecture is clearly also a factor for its success. Evaluation of reference architectures for their qualities is performed through the Architecture Tradeoff Analysis Method (designed for evaluation of software architectures) [3,37,16]. The limitations of ATAM for the context of reference architectures are addressed in [1], where extension and tailoring of ATAM for the context of reference architectures are proposed. Specific attention to the evaluation of software product line architectures is given in [74–76].

8. Conclusions

We have observed that there are different types of software reference architectures intended for use in different contexts.

Depending on their goals and application contexts, different design considerations apply. In this paper, we present a framework for reference architectures. The framework describes five main types of software reference architectures plus several variants for which the goals, context and design are in congruence. The framework provides support for the classification and analysis of existing reference architectures. Based on our experience with the framework as an analytical tool, we expect the framework to be a useful fundament for the design of successful reference. An additional benefit of this work is that it can be used as a facilitation tool for the transformation of reference architectures from one type to another. We have evaluated the framework by applying it for the analysis of 24 software reference architectures.

Elaboration of further details for each of the types (and variants) of reference architectures is a direction for future work. As part of the future work, the framework has to be applied and evaluated for the design and re-design of reference architectures.

Currently, reference architectures are approached mostly in an intuitive way without a clearly structured background. This paper provides such a structured background. We therefore believe that this work will contribute to the better understanding of the domain of reference architectures and to the design of more successful reference architectures.

Acknowledgements

We thank the reviewers and participants in WICSA/ECSA 2009 and most notably Rich Hilliard and David Emery who gave us directions for improving our initial publication. We thank Nathaniel Palmer for his clarifications on the WRM, Guido Urdaneta for his clarifications on the INAHL, Ahmed Hassan. for his clarifications on the RA WS, Michael Godfrey for his clarifications on the RA WB, Pavlin Staikov and Ravi Monangi for their clarifications on PanDORA, Leon Smiers and Theo Elzinga for their clarifications on CORA, Dan Smith for his clarification on GMSEC RA, Simeon van der Steen for his analysis of the GMSEC RA, and Evgeny Knutov for his clarifications on AHA.

References

[1] S. Angelov, J. Trienekens, P. Grefen, Towards a method for the evaluation of reference architectures: experiences from a case, in: R. Morrison, D. Balasubramaniam, K. Falkner (Eds.), Software Architecture, Second European Conference, ECSA 2008 Paphos, Cyprus, September 29–October 1, 2008 Proceedings, vol. 5292, Springer, Berlin/Heidelberg, 2008, pp. 225–240.
[2] G. Muller, A reference architecture primer, Gaudi project, 2008.
[3] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, second ed., Addison-Wesley Professional, 2003.
[4] R. Taylor, N. Medvidovic, E. Dashofy, Software Architecture: Foundations, Theory, Practice, John Wiley & Sons, 2009.
[5] S. Angelov, P. Grefen, D. Greefhorst, A classification of software reference architectures: analyzing their success and effectiveness, Software Architecture, 2009 and European Conference on Software Architecture. WICSA/ECSA 2009. Joint Working IEEE/IFIP Conference on, 14–17 September 2009, Cambridge, United Kingdom, IEEE, 2009, pp. 141–150.
[6] A. Fattah, Enterprise reference architecture: addressing key challenges facing EA and enterprise-wide adoption of SOA. Via Nova Architectura, 2009. <http://www.via-nova-architectura.org/magazine/magazine/enterprise-reference-architecture.html>.
[7] D. Greefhorst, P. Grefen, E. Saaman, P. Bergman, W. van Beek, Referentie-architectuur: off-the-shelf architectuur, Landelijk Architectuur Congres 2008, Nieuwegein, Netherlands, NAF & SDU, 2008.
[8] D. Greefhorst, P. Grefen, E. Saaman, P. Bergman, W. van Beek, Herbruikbare architectuur – een definitie van referentie-architectuur, Informatie (2009) 8–14.
[9] P. Kruchten, The Rational Unified Process: An Introduction, Addison-Wesley, 2000.
[10] L. Zhu, M. Staples, V. Tosic, On creating industry-wide reference architectures, in: EDOC '08: Proceedings of the 2008 12th International IEEE Enterprise Distributed Object Computing Conference, IEEE Computer Society, Washington, DC, USA, 2008, pp. 24–30.
[11] W. Tracz (Ed.), Domain-Specific Software Architecture (DSSA) – Frequently Asked Questions (FAQ), ACM SIGSOFT Software Engineering Notes, vol. 19, 1994, pp. 52–56.
[12] P. Avgeriou, U. Zdun, Architectural patterns revisited – a pattern language, in: Proceedings of the 10th European Conference on Pattern Languages of Programs (EuroPLoP 2005), Irsee, Germany, 2005, pp. 1–39.
[13] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, M. Stal, Pattern-Oriented Software Architecture, vol. 1: A System of Patterns, John Wiley & Sons, 1996.
[14] J. McGovern, S. Ambler, M. Stevens, J. Linn, V. Sharan, E. Jo, The Practical Guide to Enterprise Architecture, Prentice Hall PTR, 2003.
[15] I. Gorton, Essential Software Architecture, Springer-Verlag, 2006.
[16] B. Gallagher, Using the architecture tradeoff analysis method to evaluate a reference architecture: a case study, CMU/SEI-2000-TN-007 SEI, Carnegie Mellon University, CMU SEI Technical Reports, 2000.
[17] M. Anastasopoulos, J. Bayer, O. Flege, C. Gacek, A process for product line architecture creation and evaluation: PuLSE-DSSA. IESE-Report 038.00/E, Kaiserslautern, Germany, Fraunhofer IESE, 2000.
[18] F.v.d. Linden, K. Schmid, E. Rommes, Software Product Lines in Action: The Best Industrial Practice in Product Line Engineering, Springer, 2007.
[19] M. Pinzger, H. Gall, J.-F. Girard, J. Knodel, C. Riva, W. Pasman, C. Broerse, J.G. Wijnstra, Architecture recovery for product families, in: F.v.d. Linden (Ed.), Software Product-Family Engineering, 5th International Workshop, PFE 2003, Siena, Italy, November 4–6, 2003. Revised Papers, vol. 3014/2004. Springer, Berlin/Heidelberg, 2004, pp. 332–351.
[20] J. Bayer, D. Ganesan, J.-F. Girard, J. Knodel, R. Kolb, K. Schmid, Definition of reference architectures based on existing systems. IESE-WP2.6. 2003. Kaiserslautern, Fraunhofer IESE.
[21] M. Jazayeri, A. Ran, F.v.d. Linden, Software Architecture for Product Families: Principles and Practice, Addison-Wesley, 2000.
[22] K. Pohl, G. Böckle, F.v.d. Linden, Software Product Line Engineering: Foundations, Principles, and Techniques, Springer, 2005.
[23] M. Janota, J. Kiriny, G. Botterweck, Formal methods in software product lines: concepts, survey, and guidelines. Lero-TR-SPL-2008-02, Lero, University of Limerick. Lero Technical Report, 2008.
[24] D. Hollingsworth, The Workflow Reference Model, TC00-1003, Workflow Management Coalition, Workflow Management Coalition Documents, 2008.
[25] Object Management Group, Common Object Request Broker Architecture: core specification, Version 3.0.3, Object Management Group, Inc., 2004.
[26] H. Zimmermann, OSI reference model – the IS0 model of architecture for open systems interconnection, IEEE Transactions on Communications 28 (1980) 425–432.
[27] E. Mettala, M. Graham, (Eds.), The Domain-specific Software Architecture Program, Special Report CMU/SEI-92-SR-009, SEI, Carnegie Mellon University, 1992.
[28] W. Tracz, DSSA (Domain-Specific Software Architecture) – pedagogical example, ACM SIGSOFT Software Engineering Notes 20 (1995) 49–62.
[29] L. Barroca, J. Hall, P. Hall, An introduction and history of software architectures, components, and reuse, Software Architectures – Advances and Applications, Springer Verlag, 2000, pp. 1–11.
[30] F. Hayes-Roth, Architecture-based acquisition and development of software guidelines and recommendations from the ARPA Domain-Specific Software Architecture (DSSA) program, Technical report, Version 1.01, Teknowledge Federal Systems, 1994.
[31] ICTU, Kenniscentrum, and Architecture department, NORA explained: brief explanation of the Dutch government reference architecture [NORA], 2007.
[32] L. Zhu, B. Thomas, LIXI valuation reference architecture and implementation guide, Technical report, LIXI, 2007.
[33] D. Greefhorst, H. Koning, H. Vliet, The many faces of architectural descriptions, Information Systems Frontiers 8 (2006) 103–113.
[34] P. Grefen, Introduction to Corporate Information System Architecture, Internal Report, Eindhoven University of Technology, 2010.
[35] M. Metcalfe, Reading Critically at University, Sage Publications Ltd., 2006.
[36] J. Sowa, J. Zachmann, Extending and formalizing the framework for Information Systems architecture, IBM Systems Journal 31 (1992) 590–616.
[37] P. Clements, R. Kazman, M. Klein, Evaluating Software Architectures: Methods and Case Studies, Addison-Wesley Professional, 2002.
[38] L. Bratthall, P. Runeson, A taxonomy of orthogonal properties of software architecture, in: Proceedings of the Second Nordic Software Architecture Workshop (NOSA'99), 1999.
[39] N. Medvidovic, R. Taylor, A classification and comparison framework for software architecture description languages, IEEE Transactions on Software Engineering 26 (2000) 70–93.
[40] SPARC-DBMS Study Group, Interim Report: ANSI/X3/SPARC Study Group on Data Base Management Systems 75-02-08, FDT – Bulletin of ACM SIGMOD, vol. 7, 1975, pp. 1–140.
[41] OATH, OATH Reference Architecture, Release 2.0, Initiative for Open AuTHentication (OATH), 2007.
[42] N. Sauve, Towards architectural maturity: a starting point for architectural documentation of PinkRoccade Local Government's software product line, M.Sc. Thesis, Eindhoven University of Technology, 2008.
[43] J. Fessler, GMSEC Architecture Document 2.7.1. Greenbelt, Maryland, Goddard Space Flight Center, 2011.
[44] P. Grefen, R. Remmerts de Vries, A reference architecture for workflow management systems, Data & Knowledge Engineering 27 (1998) 31–57.
[45] A. Grosskurth, M. Godfrey, A reference architecture for web browsers, in: Proceedings of the 21st IEEE International Conference on Software Maintenance, IEEE Computer Society, Washington, USA, 2005, pp. 661–664.

[46] A. Hassan, R. Holt, A reference architecture for web servers, in: Proceedings of the Seventh Working Conference on Reverse Engineering (WCRE'00), IEEE Computer Society, Washington, DC, USA, 2000, pp. 150–159.
[47] G. Urdaneta, J. Colmenares, N. Queipo, N. Arapé, C. Arévalo, M. Ruz, H. Corzo, A. Romero, A reference software architecture for the development of industrial automation high-level applications in the petroleum industry, Computers in Industry 58 (2007) 35–45.
[48] Microsoft, Application architecture for .NET: designing applications and services, Microsoft, 2002.
[49] B. Paulsen, K. Babb, PanDOORA Practitioner's Guide, IBM, 2003.
[50] T. Elzinga, J. van der Vlies, L. Smiers, The CORA model, Sdu Uitgevers b.v., The Hague, 2009.
[51] D. Greefhorst, P. Gehner, Achmea streamlines application development and integration, Via Nova Architectura, 2006.
[52] D. Greefhorst, Een applicatie-architectuur voor het web bij de bank – de pro's en contra's van toestandsloosheid, Software Release Magazine 2 (1999).
[53] B. Leiner, The NCSTRL approach to open architecture for the confederated digital library, D-Lib Magazine 4 (1998).
[54] S. Angelov, P. Grefen, An e-contracting reference architecture, The Journal of Systems and Software 81 (2008) 1816–1844.
[55] H. Wu, A Reference Architecture for Adaptive Hypermedia Applications, Ph.D. thesis, Eindhoven University of Technology, 2002.
[56] A. Norta, Exploring Dynamic Inter-Organizational Business Process Collaboration, Ph. D. Thesis, Eindhoven university of Technology, 2007.
[57] F. Kuhl, R. Weatherly, J. Dahmann, Creating Computer Simulation Systems: An Introduction to the High Level Architecture, Prentice Hall PTR, Upper Saddle River, 1999.
[58] G. Klein, P. Sottile, F. Endsleff, Another HISA-the new standard: health informatics-service architecture, Studies in Health Technology and Informatics 129 (2007) 478–482.
[59] AUTOSAR, Layered Software Architecture, version 2.2.2, AUTOSAR, 2006.
[60] M. Henning, Rise and Fall of CORBA, Queue, vol. 4, 2006.
[61] Modeling and Simulation Coordination Office, DoD Interpretations of the IEEE 1516-2000 series of standards, IEEE Std 1516-2000, IEEE Std 1516.1-2000, and IEEE Std 1516.2-2000: Release 2, M&S CO, 2003.
[62] J. Dahmann, R. Fujimoto, R. Weatherly, The DoD high level architecture: an update, in: D. Medeiros, E. Watson, J. Carson, M. Manivannan (Eds.), Simulation Conference Proceedings, 1998. Winter, IEEE Computer Society Press, Los Alamitos, 1998, pp. 797–804.
[63] D. Tsichritzis, A. Klug, The ANSI/X3/SPARC DBMS framework report of the study group on database management systems, Information Systems 3 (1978) 173–191.
[64] Wikipedia, ANSI-SPARC Architecture, Wikipedia, 2010.
[65] E. Brown, A. Cristea, S. Stewart, T. Brailsford, Patterns in authoring of adaptive educational hypermedia: a taxonomy of learning styles, Educational Technology & Society 8 (2005) 77–90.
[66] L. Fischer, Workflow Handbook 2004, Future Strategies Inc., 2004.
[67] G. Hura, M. Singhal, Data and Computer Communications: Networking and Internetworking, CRC, 2001.
[68] J. Meier, A. Homer, D. Hill, J. Taylor, P. Bansode, L. Wall, R. Boucher, A. Bogawata, Application architecture guide 2.0: designing applications on the .NET platform, Microsoft Corporation, 2008.
[69] B. Paulsen, K. Babb, PanDOORA: application accelerators for e-business, Release 3.9.3 (internal presentation), IBM, 2004.
[70] K. Maly, M. Zubair, H. Anan, D. Tan, Y. Zhang, Scalable Digital Libraries Based on NCSTRL/Dienst, in: J. Borbinha, T. Baker (Eds.), Research and Advanced Technology for Digital Libraries, 4th European Conference, ECDL 2000 Lisbon, Portugal, September 18–20, 2000 Proceedings, vol. 1923. Springer-Verlag, Berlin Heidelberg, 2000, pp. 168–179.
[71] P. Reed, Reference Architecture: The best of best practices. 2002.
[72] P. Clements, L. Northrop, Software Product Lines: Practices and Patterns, Addison-Wesley Professional, Boston, Massachusetts, 2001.
[73] F. Ahmed, L.F. Capretz, The software product line architecture: an empirical investigation of key process activities, Information and Software Technology 50 (2008) 1098–1113.
[74] M. Barbacci, P. Clements, A. Lattanze, L. Northrop, W. Wood, Using the Architecture Tradeoff Analysis Method (ATAM) to evaluate the software architecture for a product line of avionics systems: a case study. CMU/SEI-2003-TN-012, SEI, Carnegie Mellon University, 2003.
[75] T. Kim, I. Ko, S. Kang, D. Lee, Extending ATAM to assess product line architecture, in: Proceedings of the 8th IEEE International Conference on Computer and Information Technology, Sydney, Australia, 8–11 July, 2008, 2008, pp. 790–797.
[76] F. Olumofin, V. Misic, A holistic architecture assessment method for software product lines, Information and Software Technology 49 (2007) 309–323.